Change Point Detection and Concept Drift in Time Series

CEFET/RJ

Data Analytics Lab

dal.global@grupo.cefet-rj.br | https://eic.cefet-rj.br/dal

Why detecting changes matters



Model validity

Forecasting and pattern discovery models are only valid while the observed process remains similar.



Control and alerting

Changes affect control, monitoring, alerting, and model adaptation. Early detection enables corrective action.



Temporal localization

The moment of change can be just as important as the existence of the change. Knowing when it occurred allows the cause to be reconstructed.

Change point vs concept drift

Change point

An observation that marks the transition between distinct dynamics in the series.

The change may involve distribution, trend, mean, variance, volatility, or autocorrelation.

When the change is gradual, the exact point may be ambiguous.

Focus: locating the moment or interval at which the series dynamics change.

Concept drift

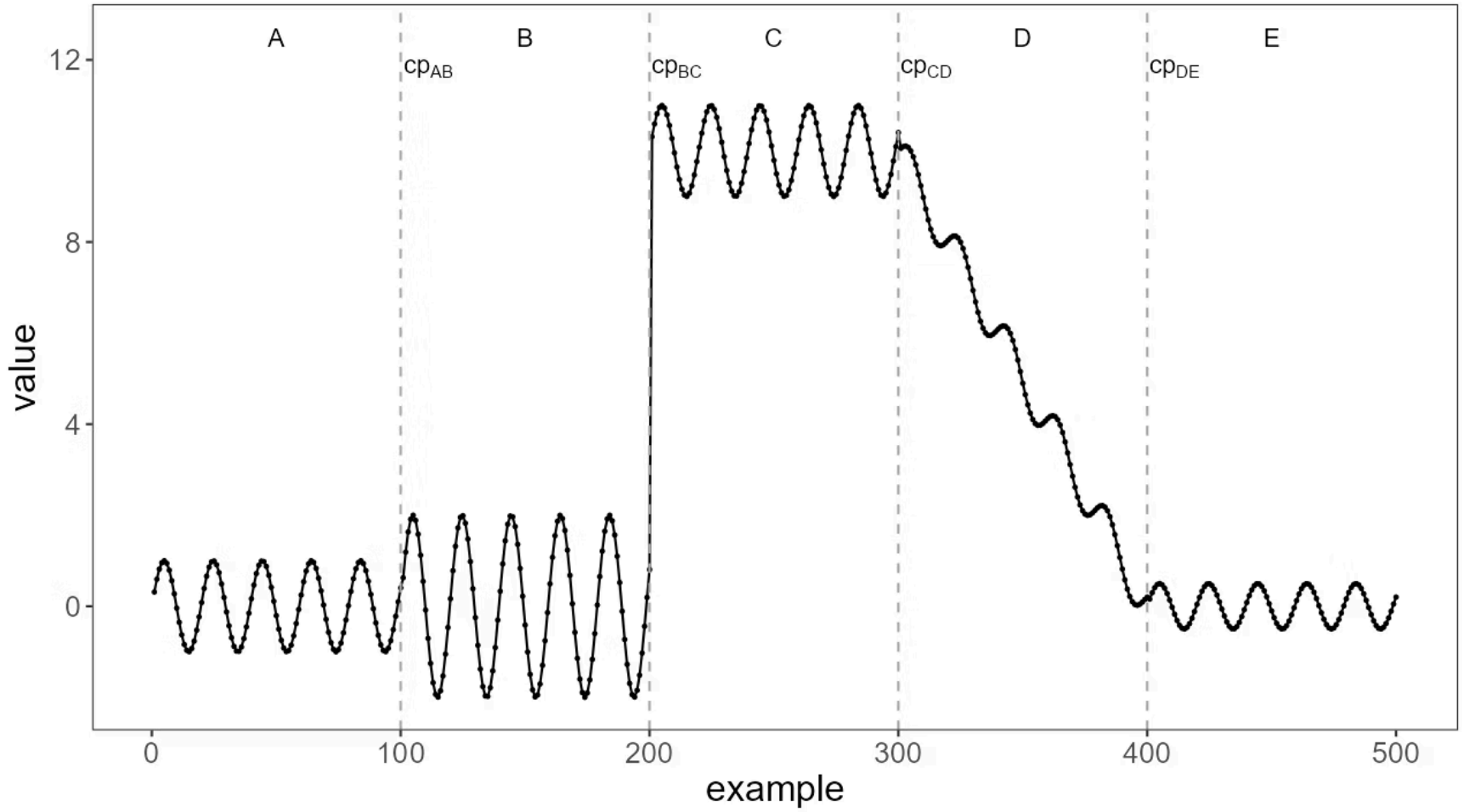
A change in the concept learned or monitored over time.

Affects the validity of models trained on historical data.

Focus: monitoring changes that affect the relationship between data and model.

In practice: change point emphasizes segmentation; drift emphasizes model validity and adaptation.

Synthetic example with four change points



The series is divided into five segments, with changes in volatility, trend, and regime.

Distinct Segments and Dynamics

- Each segment can be interpreted as having its own temporal dynamics.
- Detection seeks boundaries that make segments internally more homogeneous.
- The challenge is separating real changes from local fluctuations.

> "A good segment is one where a single model explains all internal points well."

Internal Homogeneity

Within each segment, the distribution or model should be stable. Low intra-segment variance.

Inter-segment Heterogeneity

Adjacent segments should be statistically distinct. High inter-segment variance.

Parsimony

Use the smallest number of segments that adequately explains the data. Penalization for excess change points.

Types of Change

Abrupt changes

Sudden transition between regimes.

Change point is easier to locate.

Common in failures, interventions, and structural breaks.

Gradual changes

Transition spread over an interval.

The exact moment is less evident.

May involve trend, drift, or volatility.

Dimensions of the detection problem

Offline vs online

Offline

The series has already been collected.

The goal is to explain when the dynamics changed.

Can use information before and after the change.

Methods: AMOC, BinSeg, PELT, Chow Test, GFT/CUSUM.

Online

Observations arrive as a stream.

The decision must be fast and incremental.

Trade-off between detection delay and false alarm.

Methods: CUSUM, DDM, ADWIN, Page-Hinkley.

Number of change points

Known

The problem is to locate m changes.

More common in controlled offline analysis.

Allows formulating a search for optimal partitions.

Unknown

The method must decide how many segments exist.

Over-detection generates false positives.

Under-detection generates false negatives.

Penalization (BIC, AIC) controls the trade-off.

Taxonomy of Change Point Detection

Taxonomy Outline

Segmentation Search

- Objective: minimize total cost across all segments.
- Penalization controls the number of change points (BIC, AIC, β).
- AMOC: single change via likelihood ratio.
- BinSeg: recursive AMOC, greedy splitting.
- PELT: dynamic programming with pruning, optimal solution in O(n).

Regression Search

- Evaluates breakpoints via models fitted per segment.
- Criterion: RSS reduction or log-likelihood gain.
- Chow Test: F-statistic for structural break in linear regression.
- GFT/CUSUM: accumulated residuals from a fitted model.

Sliding Window Search

- Analyzes fixed-size subsequences along the series.
- Does not require the full series; applicable in semi-online mode.
- SCP: fixed seminal point at the center of the window, line comparison.
- Change Finder: two-phase AR with score smoothing.

Segmentation Search

Principle

Objective

Looks for partitions that make each segment internally coherent. Minimizes a global cost function over all segments.

Cost function

Can minimize intra-segment distance (variance, squared error) or maximize likelihood per segment.

Penalization

To avoid over-segmentation, adds a penalty for the number of change points (BIC, AIC, linear penalty).

Methods in this family

Exact search

- Dynamic programming guarantees the optimal solution.
- PELT uses pruning for O(n) efficiency.
- Cost: depends on the cost function and the number of change points.

Approximate search

- BinSeg applies recursive greedy splitting.
- Does not guarantee a global optimum, but is efficient and practical.
- Useful when n is large and m is small.

Regression-based Search

Principle

Objective

Evaluates breakpoints through models fitted to segments. A good breakpoint reduces error or improves goodness of fit.

Evaluation Criterion

Compares RSS (residual error) or log-likelihood of the single model with the sum of per-segment models.

Scanning

For each candidate k, fits models before and after and computes the gain. The optimal k maximizes the gain.

Methods in this family

Chow Test

- Tests structural break in a linear regression model.
- Uses F-statistic to compare RSS of the single model with ESS of the segments.
- Requires normality assumption on residuals.

GFT (CUSUM of Residuals)

- Monitors cumulative residuals of a fitted model.
- Detects systematic deviation of residuals from the expected value.
- Does not require a fixed candidate k: locates the peak of the cumulative process.

Sliding Window Search

Principle

Objective

Analyzes fixed-size subsequences along the series. Within the window, it tests whether there is a relevant difference between before and after a candidate point.

Advantage

Does not require the entire series to be available. Can be applied in online or semi-online mode.

Limitation

The window size affects sensitivity. Small windows detect rapid changes; large windows detect gradual changes.

Methods in this family

SCP

- Compares regressions within a sliding window.
- Evaluates whether the internal split of the window reduces the fitting error.
- Suitable for structural changes in linear models.

Change Finder

- Uses an AR model to score outliers and then detects changes in the scores.
- Two phases: outlier detection and change point detection.
- Robust to isolated outliers thanks to moving average smoothing.

AMOC: Intuition

AMOC: Intuition

AMOC searches for a single change across the entire series. Each candidate k splits X into two segments. The question is whether the two segments explain the data better than a single regime.

$$X_{1:n} \to X_{1:k} + X_{k+1:n}$$

Monitored statistic: likelihood gain when splitting at k. Parameters: threshold γ (gamma).

Pseudocode

INPUT: series $X_{1:n}$, threshold γ
PRE-CONDITION: $n \geq 2$; at most one dominant change is expected

STEP 1 — For each candidate k from 2 to n−1:
  compute $L_k = \text{logLik}(X_{1:k}) + \text{logLik}(X_{k+1:n})$
STEP 2 — Compute $L_0 = \text{logLik}(X_{1:n})$ without segmentation
STEP 3 — Compute statistic:
  $\Lambda = 2 \cdot (\max_k L_k - L_0)$
STEP 4 — If $\Lambda >$ γ:
  $k^* = \text{argmax}_k L_k$
  return k^* as change point
  otherwise: return "no change"

OUTPUT: position k^* or absence of change
POST-CONDITION: AMOC detects at most 1 change point

AMOC: Null and Alternative Hypothesis

H0: no change

The series is explained by a single distribution.

There is a single set of parameters $\theta 0$.

H1: change at k

The segments before and after k have different distributions.

Each segment has its own parameters $\theta 1$ and $\theta 2$.

AMOC: likelihood ratio

Test statistic and decision

Compares the best fit with a single distribution against the best fit split into two segments. The optimal k maximizes the likelihood gain. The decision depends on a threshold to reject the null hypothesis of no change.

$$\lambda = 2 \cdot (\max_{k} ML(X, k) - ML(X))$$

Under H_0, λ asymptotically follows a chi-squared distribution with degrees of freedom equal to the difference in parameters between the two models.

Interpreting the gain

Small λ

The two segments do not explain the data better than a single regime. There is no evidence of a change.

λ > threshold

The likelihood gain is statistically significant. There is evidence of a change point at k*.

Threshold selection

Can be set using the asymptotic distribution (chi-squared), by permutation, or by an information criterion (AIC/BIC).

BinSeg: Recursive AMOC

BinSeg: Intuition

BinSeg applies AMOC recursively. At each iteration, it finds the best change point in the current segment and splits. Repeats until a stopping criterion is met.

Objective function: minimize total cost $C(X_{1:k}) + C(X_{k+1:n})$ at each split.

Parameters: maximum number of change points *m* or gain threshold.

Note: greedy algorithm — does not guarantee a globally optimal solution.

Pseudocode

INPUT: series $X_{1:n}$, max. change points m (or threshold λ)
PRE-CONDITION: $n \geq 2$

STEP 1 — Initialize: segments = $\{X_{1:n}\}$, CPs = {}
STEP 2 — While $|CPs| < m$:
  For each segment S in segments:
    apply AMOC(S) → candidate k^*, gain g^*
  Select segment with highest g^*
STEP 3 — If $g^* > \lambda$:
  Split segment into S_1 and S_2 at k^*
  Add k^* to CPs
  Replace S with $\{S_1, S_2\}$ in segments
  otherwise: stop
STEP 4 — Return sorted CPs

OUTPUT: set of change points $\{k_1, k_2, \ldots, k_m\}$
POST-CONDITION: each segment is internally homogeneous

PELT: Efficient Segmentation

PELT: Intuition

PELT finds the globally optimal segmentation via dynamic programming with pruning. Pruning eliminates candidates that can never be optimal, reducing computational cost.

Objective function: minimize $\sum C(\text{segment}) + \beta \cdot |\text{CPs}|$, where β is the penalty per change point.

Parameters: cost function C (e.g., variance, negative log-likelihood), penalty β.

Complexity: O(n) under linear growth of change points.

Pseudocode

INPUT: series $X_{1:n}$, cost $C(\cdot)$, penalty β
PRE-CONDITION: C satisfies the pruning inequality

STEP 1 — Initialize: $F(0) = -\beta$, $cp(0) = \varnothing$, $R = \{0\}$
STEP 2 — For t = 1 to n:
  $F(t) = \min_{s \in R} [F(s) + C(X_{s+1:t}) + \beta]$
  $s^* = \text{argmin}_{s \in R} [F(s) + C(X_{s+1:t}) + \beta]$
  $cp(t) = cp(s^*) \cup \{s^*\}$
STEP 3 — Prune R:
  For each $s \in R$:
    If $F(s) + C(X_{s+1:t}) + \beta > F(t)$:
      remove s from R (can never be optimal)
  Add t to R
STEP 4 — Recover change points via cp(n)

OUTPUT: globally optimal set of change points
POST-CONDITION: globally optimal solution under penalty β

Chow Test: structural change

Chow Test: intuition

Based on a linear regression model: $y_t = \beta_t x_t + \omega_t$

$$y_t = \beta_t x_t + \omega_t$$

Tests whether the parameters β remain stable or change over time.

Especially useful when a change implies a structural break in the model.

H_0: $\beta_1 = \beta_2$ (equal parameters before and after k)

H_1: $\beta_1 \neq \beta_2$ (structural break at k)

Pseudocode

INPUT: series (y_t, x_t), candidate k, significance level α
PRE-CONDITION: residuals approximately normal and homoscedastic

STEP 1 — Fit full model over $X_{1:n}$:
$\hat{y}_t = \hat{\beta} \cdot x_t$
$RSS = \Sigma(y_t - \hat{y}_t)^2$
STEP 2 — Fit model on segment 1 ($t \leq k$):
$\hat{y}_t^{(1)} = \hat{\beta}_1 \cdot x_t$
$RSS_1 = \Sigma_{t \leq k} (y_t - \hat{y}_t^{(1)})^2$
STEP 3 — Fit model on segment 2 ($t > k$):
$\hat{y}_t^{(2)} = \hat{\beta}_2 \cdot x_t$
$RSS_2 = \Sigma_{t > k} (y_t - \hat{y}_t^{(2)})^2$
$ESS = RSS_1 + RSS_2$
STEP 4 — Compute F-statistic:
$F = [(RSS - ESS)/r] / [ESS/(n - 2r)]$
where r = number of parameters
STEP 5 — If $F > F_{critical}(\alpha)$: reject H_0 → change at k

OUTPUT: structural break decision at k
POST-CONDITION: repeat for multiple k (scanning)

Chow Test: F-statistic

F-statistic and interpretation

Fits a model for the entire series and separate models before/after the candidate point. Compares RSS and ESS to assess whether the split significantly reduces error. A high F indicates evidence of structural change.

$$F = \frac{(RSS - ESS)}{ESS/(n - 2r)}$$

Where **RSS** = residual sum of squares of the single model; **ESS** = $RSS_1 + RSS_2$ of the separate models; **r** = number of parameters; **n** = series length.

Under H_0, F ~ F(r, n−2r). Reject H_0 if F > F_critical(α).

Interpretation of components

RSS (single model)

Residual error when fitting a single model to the entire series. Represents the cost of ignoring the possible break.

ESS (separate models)

Sum of residual errors from the two segments. Represents the cost of allowing the break at k.

Fit gain

RSS − ESS measures how much the split reduces error. Normalized by ESS/(n−2r) to control for degrees of freedom.

GFT/CUSUM: Cumulative Residuals

GFT/CUSUM: Intuition

Fits a regression model and monitors the cumulative residuals. Under stability, the cumulative process stays within expected bounds. Under a structural change, it displays a peak near the change point.

Monitored statistic:

$$W_n^0(u) = \frac{1}{\hat{\sigma}\sqrt{n}} \cdot \sum_{i=1}^{\lfloor nu \rfloor} \hat{\omega}_i$$

Parameters: baseline regression model, significance threshold.

Pseudocode

INPUT: series $X_{1:n}$, regression model M, threshold h
PRECONDITION: M fitted on a stable reference period

STEP 1 — Fit M over $X_{1:n}$ → obtain residuals $\hat{\omega}_1, \ldots, \hat{\omega}_n$
STEP 2 — Estimate standard deviation $\hat{\sigma}$ of residuals
STEP 3 — For each $u \in (0,1]$:
  Compute cumulative process:
  $W(u) = (1 / \hat{\sigma}\sqrt{n}) \cdot \Sigma_{i=1}^{\lfloor nu \rfloor} \hat{\omega}_i$
STEP 4 — Compute test statistic:
  $\sup_u |W(u)|$ or functional variant
STEP 5 — If statistic > h:
  Locate $u^* = \text{argmax}_u |W(u)|$
  Report change point at $\lfloor nu^* \rfloor$
  otherwise: series is stable

OUTPUT: change point position or absence of change
POSTCONDITION: sensitive to structural breaks in the model

SCP: sliding window strategy

Intuition

SCP slides a fixed-size window over the series. The center point of each window is the fixed candidate — the seminal point. It compares the error of a linear regression over the entire window with the error of two separate regressions: before and after the center. The difference in errors is the change score. Change points are the scores identified as outliers by the boxplot filter.

Monitored statistic:

$$\text{score}_t = \overline{|r_{\text{full}}|} - \overline{|r_{\text{split}}|}$$

Parameters:

- sw_size: window size (default 30)
- offset = ⌊sw_size / 2⌋: seminal point position
- Automatic threshold: Q3 + 1.5 · IQR

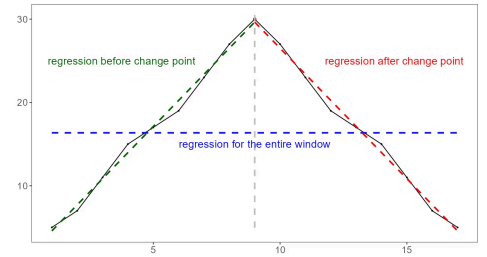
Pseudocode

INPUT: series $X_{1:n}$, sw_size
PRE-CONDITION: sw_size ≥ 4; offset = ⌊sw_size / 2⌋

STEP 1 — For each window W of size sw_size:
  Fit a line over the entire W
  err_full = mean of |residuals|
STEP 2 — Split W at the seminal point (position offset):
  Fit a line on W[1 : offset−1] → err_left
  Fit a line on W[offset+1 : sw_size] → err_right
  err_split = mean of (err_left + err_right)
STEP 3 — Compute window score:
  score_t = err_full − err_split
STEP 4 — After scanning all windows:
  threshold = Q3(scores) + 1.5 · IQR(scores)
  change_point = {t : score_t > threshold}
STEP 5 — Align indices:
  pad with zeros at borders (offset on the left,
  sw_size − offset − 1 on the right)

OUTPUT: positions t where score_t > threshold
POST-CONDITION: each CP is the center of the window
  with the highest local fit contrast

Illustration



Compares the fit of a single regression with separate regressions before and after the candidate point.

Change Finder: Two Phases

Change Finder: Intuition

Detects change points in two phases. Phase 1: models the series with AR and scores outliers via log-likelihood. Phase 2: smooths the scores and applies a second AR model to detect changes in the score pattern.

Monitored statistic: Phase 1: outlier score $s_t = -\log p(x_t \mid x_{t-1}, \ldots, x_{t-p})$, smoothed as $\bar{s}_t$. Phase 2: score $a_t = -\log p(\bar{s}_t \mid \bar{s}_{t-1}, \ldots, \bar{s}_{t-p})$, smoothed as $\bar{a}_t$.

Parameters: AR order p, smoothing window w, alarm threshold.

Pseudocode

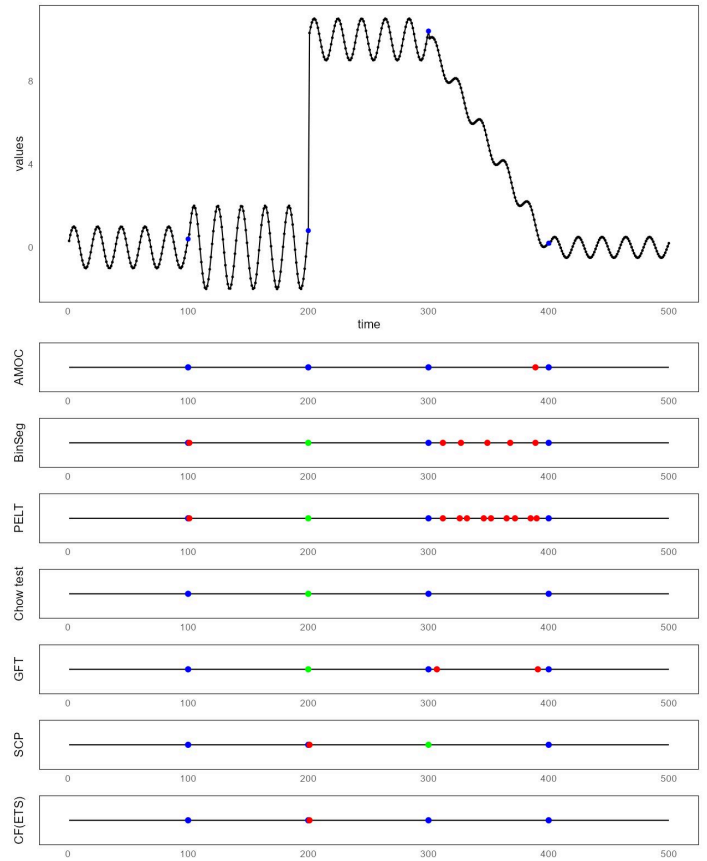
INPUT: series $X_{1:n}$, AR order p, window w, threshold h
PRE-CONDITION: n > p + w

PHASE 1 — Outlier Detection:
STEP 1 — Fit AR(p) over $X_{1:n}$
STEP 2 — For each t > p:
  $s_t = -\log p(x_t \mid x_{t-1}, \ldots, x_{t-p})$
STEP 3 — Smooth: $\bar{s}_t = (1/w) \cdot \Sigma_{i=0}^{w-1} s_{t-i}$

PHASE 2 — Change Point Detection:
STEP 4 — Fit AR(p) over $\bar{s}_{1:n}$
STEP 5 — For each t:
  $a_t = -\log p(\bar{s}_t \mid \bar{s}_{t-1}, \ldots, \bar{s}_{t-p})$
STEP 6 — Smooth: $\bar{a}_t = (1/w) \cdot \Sigma_{i=0}^{w-1} a_{t-i}$
STEP 7 — If $\bar{a}_t > h$: flag change point at t

OUTPUT: time steps t where $\bar{a}_t > h$
POST-CONDITION: robust to isolated outliers

Comparison of change point detectors



Green indicates a correct detection, blue indicates a missed event, and red indicates an incorrect detection.

What is concept drift

Definition

It is a change in the learned or monitored concept over time. It affects the validity of models trained on historical data.

Origin

It can arise from changes in user behavior, the environment, policies, or the collected data itself.

Consequence

The model starts making more errors without its internal parameters changing. The degradation can be gradual or abrupt.

Difference from change point

Change point focuses on the time series itself. Concept drift focuses on the relationship between data and the learned concept — especially in predictive models.

When to monitor

Whenever a model is deployed in production and data keeps coming in. Any ML system in production is subject to concept drift.

Concept as Distribution and Drift

Concept as Distribution

Consider data D organized in batches.

The concept in a batch is represented by the distribution associated with the observed data.

Concept change means a statistically relevant change in this distribution.

$$concept(SD_i) = p(\chi_i)$$

Drift Between Batches

Between consecutive batches, drift appears when the distributions are no longer equivalent.

The difference can occur in the input data, in the X → Y relationship, or in the model's error.

$$p(\chi_i) \neq p(\chi_{i+1})$$

Virtual drift (input drift)

Changes p(X). The X→Y relationship may remain valid.

Real drift

Changes p(Y|X). The model loses predictive validity and requires adaptation.

Concept Drift Taxonomy

Drift Types

- **Virtual:** changes p(X), the X→Y relationship may remain valid.
- **Real:** changes p(Y|X), model loses predictive validity.
- Abrupt, gradual, incremental, recurring.

Detector Types

- **Error-based:** monitor model performance (DDM, HDDM, ECDD).
- **Distribution-based:** compare data windows (ADWIN, KSWIN, KLD, MCDD).

Methods

- **Incremental:** CUSUM, Page-Hinkley, DDM, ADWIN.
- **Batch-based:** MCDD, KLD, KSWIN.
- **Ensemble:** combine multiple detectors or models.

Treatment

- Full retraining after alarm.
- Continuous incremental update.
- Adaptive ensemble with performance-based weighting.

The taxonomy organizes drift types, detector types, methods, and treatment approaches.

Real vs virtual drift

Virtual drift

The distribution of input data changes.

The relationship with the target variable may remain valid.

Can affect representativeness and sampling.

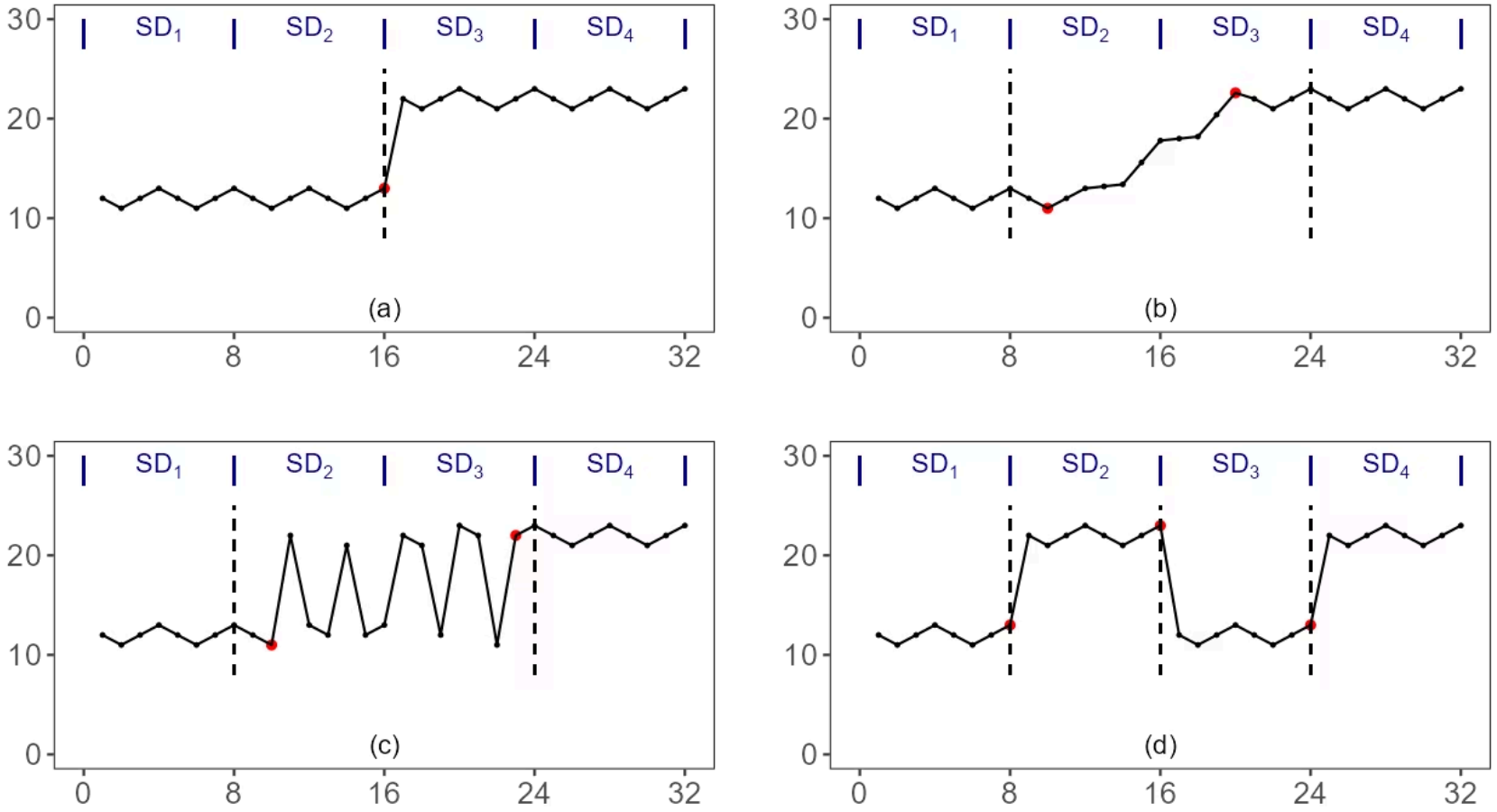
Real drift

The relationship between input and output changes.

The model loses predictive validity.

Typically requires adaptation or retraining.

Types of drift



Drifts can be abrupt, incremental, gradual, or recurring.

Detecting change vs adapting the model

Detect drift

- Answers: has the concept changed?
- Monitors stream statistics or model error.
- Produces an alarm signal with temporal location.
- Sensitivity vs specificity: central trade-off.

Adapt the model

- Answers: what to do after the change?
- May involve full retraining, incremental update, or model selection.
- Depends on good detection to avoid unnecessary adaptation.

Overly sensitive detector

Causes false alarms. The model is retrained unnecessarily, absorbing noise as if it were signal.

Overly conservative detector

Reacts too late. The model operates with an outdated concept, accumulating error before the alarm.

Ideal balance

Calibrate the alarm threshold based on acceptable latency, retraining cost, and false alarm risk.

Types of Drift Detectors

Error-based

Monitor drops in model performance.

DDM, HDDM, CUSUM and ECDD follow this approach.

Require labels or feedback.

Distribution-based

Compare batches, windows or samples.

MCDD, KLD, KSWIN, Page-Hinkley and ADWIN are examples.

Can act before the impact on error is observed.

Selection Criteria

Label availability

Error-based detectors require true labels (*y*t) in real time. If labels arrive with delay, prefer data-based detectors.

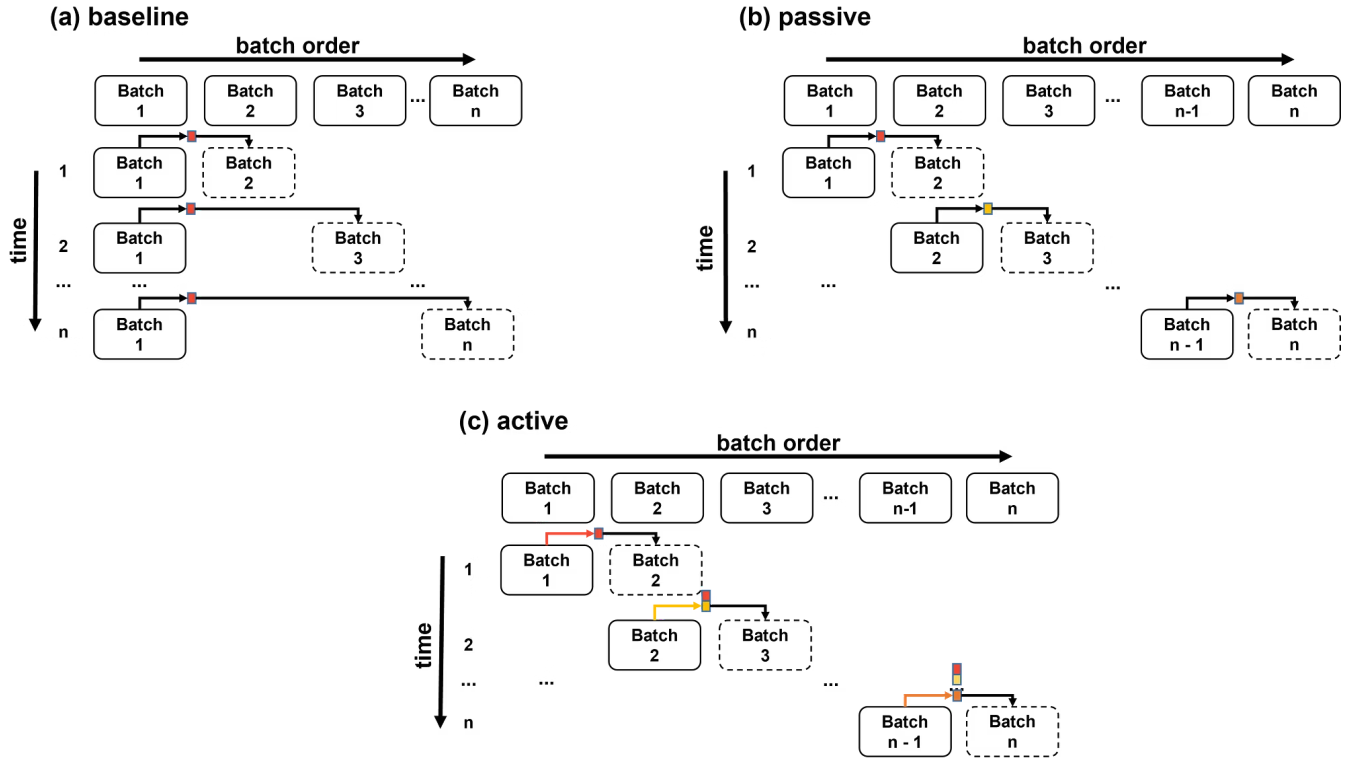
Expected drift type

Mean drift → CUSUM, Page-Hinkley. Distribution drift → KSWIN, KLD. Classifier error drift → DDM, HDDM, ECDD.

Computational cost

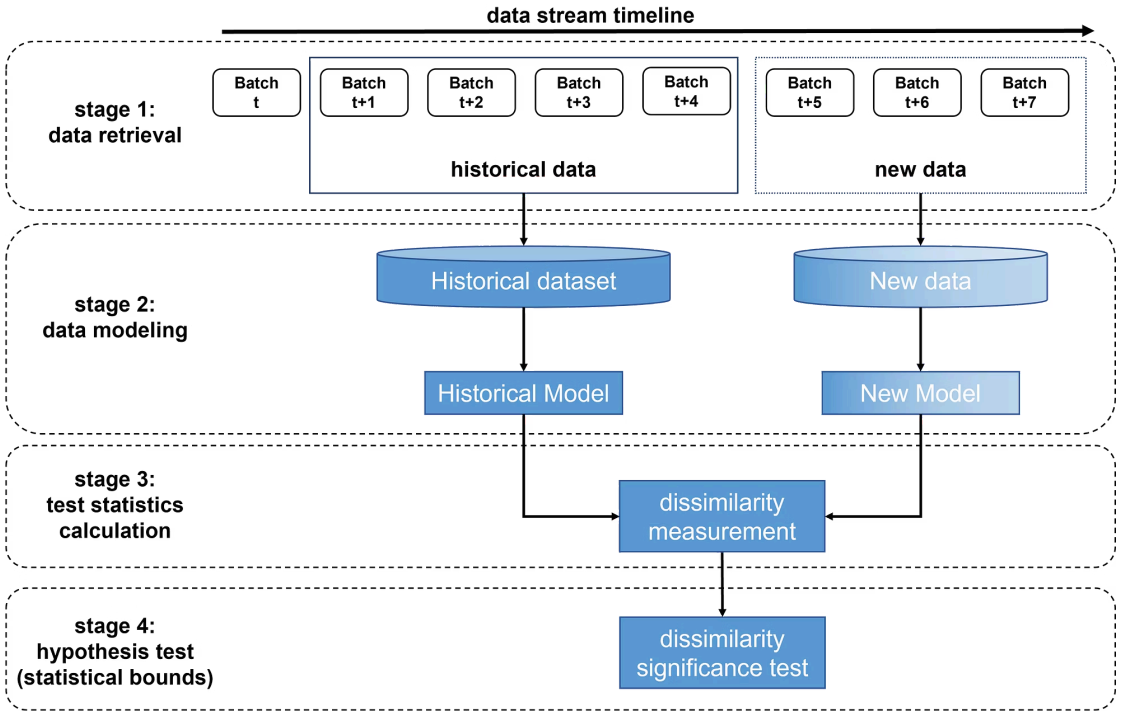
Incremental detectors (CUSUM, DDM, ADWIN) are suitable for high-speed streams. Batch detectors (MCDD, KLD) are more robust but require data accumulation.

Drift Treatment



Baseline does not react, passive updates continuously, and active updates after detection.

General drift detection framework



The framework combines streaming data, monitoring, detection, and eventual model adaptation.

When to update the model

Continuous update

- Tracks gradual changes without the need for an explicit detector.
- Risk: absorbing noise and fluctuations as if they were real changes.
- Suitable when the concept changes slowly and the retraining cost is low.

Update after alarm

- Reduces computational cost: retrains only when necessary.
- Depends on good detection to avoid reacting too late or on a false alarm.
- Suitable when the concept changes abruptly and retraining is expensive.

Adaptive ensemble

- Maintains multiple models trained on different time windows.
- Weights models by recent performance.
- Robust to different drift speeds.

Acceptable latency

How long can the model operate with an outdated concept before causing harm?

Retraining cost

Is retraining expensive? Prefer incremental updates or an ensemble approach.

False alarm risk

A false alarm leads to unnecessary retraining. Calibrate the detector to the context.

DDM and HDDM

DDM and HDDM: Intuition

DDM: monitors the error rate p_t and its standard deviation $s_t = \sqrt{(p_t(1-p_t)/t)}$. Alarm when $p_t + s_t > p_min + k \cdot s_min$.

HDDM: uses Hoeffding inequalities to control detection with probabilistic bounds. More robust to transient fluctuations.

DDM parameters: *k_warning* (warning level), *k_alarm* (alarm level).

HDDM parameters: *δ_warning*, *δ_alarm* (type I error probabilities).

INPUT: prediction stream ($\hat{y}_t$, y_t), parameters k_w, k_a
PRE-CONDITION: classifier trained on initial concept

STEP 1 — Initialize: p_min = ∞, s_min = ∞, t = 0
STEP 2 — For each new observation ($\hat{y}_t$, y_t):
  e_t = 1 if $\hat{y}_t \neq y_t$, else 0
  Update p_t = cumulative mean of e_t
  Compute $s_t = \sqrt{(p_t(1 - p_t) / t)}$
STEP 3 — Update minimums:
  If $p_t + s_t <$ p_min + s_min:
    p_min = p_t, s_min = s_t
STEP 4 — Check thresholds:
  If $p_t + s_t >$ p_min + k_a · s_min:
    ALARM: signal drift, reset model
  If $p_t + s_t >$ p_min + k_w · s_min:
    WARNING: accumulate recent data

OUTPUT: alarm and warning timestamps
POST-CONDITION: HDDM replaces k with Hoeffding bounds

Hoeffding Bound: Intuition

Intuition and Formula

- Provides a bound on expected differences between sample estimates.
- If the observed difference exceeds the bound, there is statistical evidence of change.
- Useful in streaming settings because it allows incremental decision-making.

$$\Pr(|\bar{x} - E[x]| \geq \varepsilon) \leq 2\exp(-2n\varepsilon^2)$$

Parameters: *n* (sample size), *ε* (tolerated error margin).

Application in Drift Detection

Collect Sample

Accumulate *n* observations from the current stream.

Compute ε_n

Derive the Hoeffding threshold for *n* and desired *δ*: *ε = √(ln(1/δ) / 2n)*.

Compare Means

If |x̄_recent − x̄_reference| > ε_n, signal drift with confidence 1−δ.

CUSUM: Accumulated Evidence

CUSUM: Intuition

Accumulates successive deviations from the expected behavior. When the cumulative sum exceeds a threshold, it signals a change. Simple, incremental, and sensitive to persistent changes.

Monitored statistic:

$$z_t = \max(0,\ z_{t-1} + \omega_t - \delta)$$

Parameters:

- **δ** — minimum magnitude of change to detect
- **h** — alarm threshold

Resets $z_t = 0$ after alarm.

INPUT: stream $x_1, x_2, \ldots$, expected mean μ_0, δ, threshold h
PRE-CONDITION: stationary series under H_0

STEP 1 — Initialize: $z_0 = 0$, $t = 1$
STEP 2 — For each new observation x_t:
  Compute deviation: $\omega_t = x_t - \mu_0$
  Update statistic:
    $z_t = \max(0, z_{t-1} + \omega_t - \delta)$
STEP 3 — If $z_t > h$:
  Signal change at time t
  Reset: $z_t = 0$ (optional)
  otherwise: continue
STEP 4 — Increment t → return to Step 2
OUTPUT: time steps t where $z_t > h$
POST-CONDITION: detects persistent mean shifts

ECDD and MCDD

ECDD

Uses EWMA to smooth the error rate:

$$Z_t = \lambda \cdot e_t + (1 - \lambda) \cdot Z_{t-1}$$

Detects when the tracker moves away from the control limit.

Adds inertia to monitoring, reducing false alarms.

Parameters:

- **λ** — EWMA smoothing factor
- **L** — control limit width

Alarm when:

$$Z_t > \mu_Z + L \cdot \sigma_Z$$

MCDD

Compares recent and previous batches using multivariate statistics.

Evaluates the mean and variance of predictor variables.

Can use parametric (**Hotelling T^2**) or non-parametric (**MMD**) tests.

Parameters:

- **batch size** — comparison window
- **α** — significance level

Useful when drift affects multiple variables simultaneously.

Page-Hinkley and KSWIN

Page-Hinkley

- Variation of CUSUM that accumulates differences relative to the running mean.
- Statistic: PHt = $\Sigma_{i=1}^{t}$ ($x_i - \bar{x}_t - \delta$), alarm when $PH_t - \min(PH) > h$.
- Works well for abrupt mean changes.
- Parameters: δ (minimum magnitude), h (threshold).

—— PAGE-HINKLEY ——

INPUT: stream x_t, δ, threshold h

STEP 1 — Initialize: $PH_0 = 0$, $m_0 = 0$, $\bar{x}_0 = 0$

STEP 2 — For each x_t:

Update mean: $\bar{x}_t = \bar{x}_{t-1} + (x_t - \bar{x}_{t-1})/t$

$PH_t = PH_{t-1} + (x_t - \bar{x}_t - \delta)$

$m_t = \min(m_{t-1}, PH_t)$

If $PH_t - m_t > h$: ALARM at t

KSWIN

- Uses the Kolmogorov-Smirnov test over two sliding windows.
- Compares empirical distributions of W_r (reference) and W_n (recent).
- Does not require strong parametric assumptions.
- Parameters: window size n, sub-window size r, α (significance level).

—— KSWIN ——

INPUT: stream x_t, window n, sub-window r, level α

STEP 1 — Maintain sliding window W of size n

STEP 2 — For each new x_t:

Add x_t to W; remove oldest element

W_r = last r elements of W

W_ref = W \ W_r (reference elements)

STEP 3 — Compute KS statistic:

D = sup_x |F_ref(x) − F_r(x)|

p-value via KS distribution

STEP 4 — If p-value < α: ALARM at t

OUTPUT: alarm instants for each method

KLD: Divergence Between Distributions

Definition and Formula

- Compares an old distribution P with a recent distribution Q.
- Measures the excess surprise when using the expected distribution to explain current data.
- Drift is indicated when the divergence exceeds a threshold.

$$D_{KL}(P\|Q) = \sum P(x_i) \cdot \ln\left(\frac{P(x_i)}{Q(x_i)}\right)$$

KLD is not symmetric: D_KL(P‖Q) ≠ D_KL(Q‖P). For bidirectional detection, use Jensen-Shannon divergence.

Practical Application

Distribution Estimation

- Estimate P from a reference window (historical data).
- Estimate Q from a recent window of the stream.
- Use histograms or kernel density estimation (KDE).

Drift Decision

- Calculate D_KL(P‖Q) for each new batch.
- Compare with threshold h calibrated by bootstrap or cross-validation.
- If D_KL > h: signal drift and update P ← Q.

ADWIN: Adaptive Windowing

ADWIN: Intuition

Explanation: Maintains a window W of variable size over the stream. Splits W into two sub-windows W_0 (old) and W_1 (recent) and tests for statistical difference via Hoeffding bound. When a change is detected, discards W_0 and keeps W_1.

Monitored statistic: $|\mu(W_0) - \mu(W_1)| > \varepsilon_cut$, where ε_cut depends on the window sizes and δ.

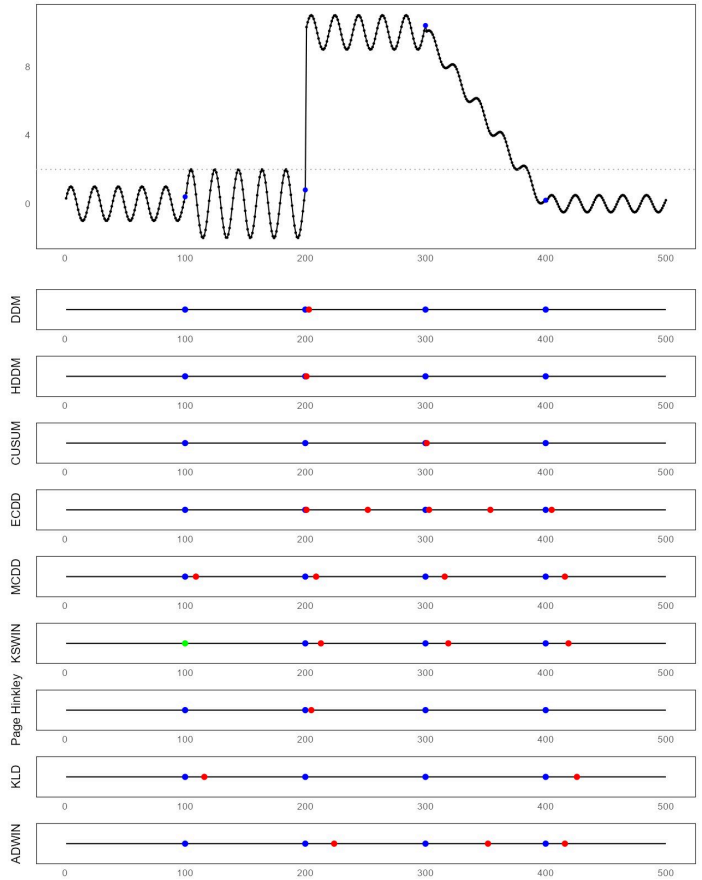
Parameters: δ (statistical confidence), minimum sub-window size.

INPUT: stream $x_1, x_2, \ldots$, confidence δ
PRE-CONDITION: $\delta \in (0,1)$

STEP 1 — Initialize: $W = []$, $t = 0$
STEP 2 — For each new observation x_t:
  Add x_t to the end of W
STEP 3 — For each split $W = W_0 \cup W_1$:
  Compute $\hat{\mu}_0 = \text{mean}(W_0)$, $\hat{\mu}_1 = \text{mean}(W_1)$
  Compute Hoeffding threshold:
    $\varepsilon_cut = \sqrt{((1/2|W_0| + 1/2|W_1|) \cdot \ln(4|W|/\delta))}$
  If $|\hat{\mu}_0 - \hat{\mu}_1| \geq \varepsilon_cut$:
    Signal drift
    Discard W_0: $W \leftarrow W_1$
    Restart search
STEP 4 — Increment $t \rightarrow$ return to Step 2

OUTPUT: drift instants and updated window W
POST-CONDITION: W always represents the most recent concept

Comparison of drift detectors



Green indicates correct detection, blue indicates a missed event, and red indicates a false detection.

Multivariate Series and Spectral Methods

Multivariate Challenges

Dimensionality

More dimensions increase the difficulty of locating and explaining changes. The curse of dimensionality affects statistical tests.

Partial Changes

Changes may occur in only a subset of variables. Univariate detectors applied separately miss correlations.

Spatial Dependence

In spatio-temporal series, changes can propagate geographically. The location has a spatial dimension beyond the temporal one.

Approaches

Dimensionality Reduction

PCA, autoencoders, and random projections compress the multivariate series. Univariate detectors are applied in the reduced space.

Multivariate Tests

Hotelling T^2, MMD (Maximum Mean Discrepancy), and energy tests compare multivariate distributions directly. More powerful, but computationally more expensive.

Spectral Methods

PCA and autoencoders capture compact representations. Drifts appear as changes in reconstruction or in the latent space. Promising for non-stationary and streaming series.

Practical Challenges

Delay

Detecting early enough to act. There is a trade-off between detection speed and false alarm rate. More sensitive detectors react faster, but also to noise.

False Alarm

Avoiding unnecessary adaptation. A false alarm leads to retraining without need, potentially degrading the model. Calibrate the threshold with validation data.

Cost

Keeping monitoring and retraining viable. In high-frequency systems, the computational cost of the detector must be compatible with the required latency.

Interpretation

Explaining what type of change occurred. Knowing whether the change is in the input, in the X→Y relationship, or in the model error guides the adaptation strategy.

Scarce Labeled Data

In production, true labels arrive with delay or not at all. Distribution-based detectors are preferable in these scenarios.

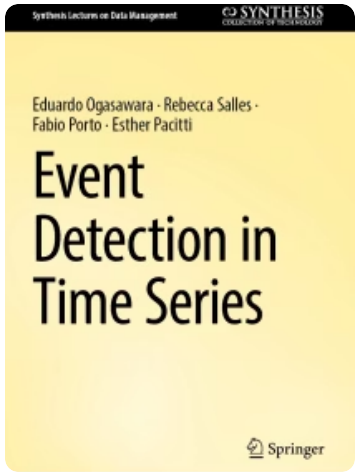
Recurring Changes

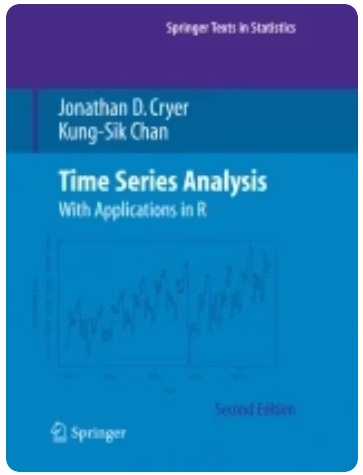
Concepts can return (concept recurrence). Storing old models and reactivating them can be more efficient than retraining from scratch.

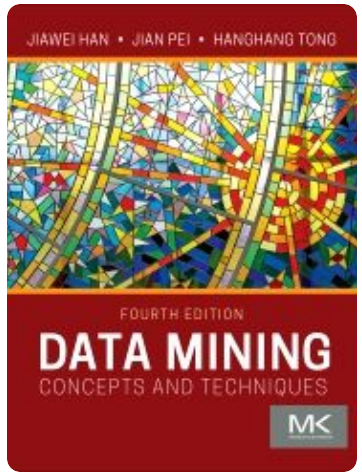
Continuous Monitoring

Implementing monitoring pipelines with automatic alerts, performance dashboards, and detection logs for auditing.

Bibliographic References







Event Detection in Time Series

Ogasawara, E.; Salles, R.; Porto, F.; Pacitti, E. Springer Nature Switzerland, 2025.

Time Series Analysis: With Applications in R

Cryer, J. D.; Chan, K.-S. Springer, 2008.

Data Mining: Concepts and Techniques

Han, J.; Pei, J.; Tong, H. Morgan Kaufmann, 4th ed., 2022.